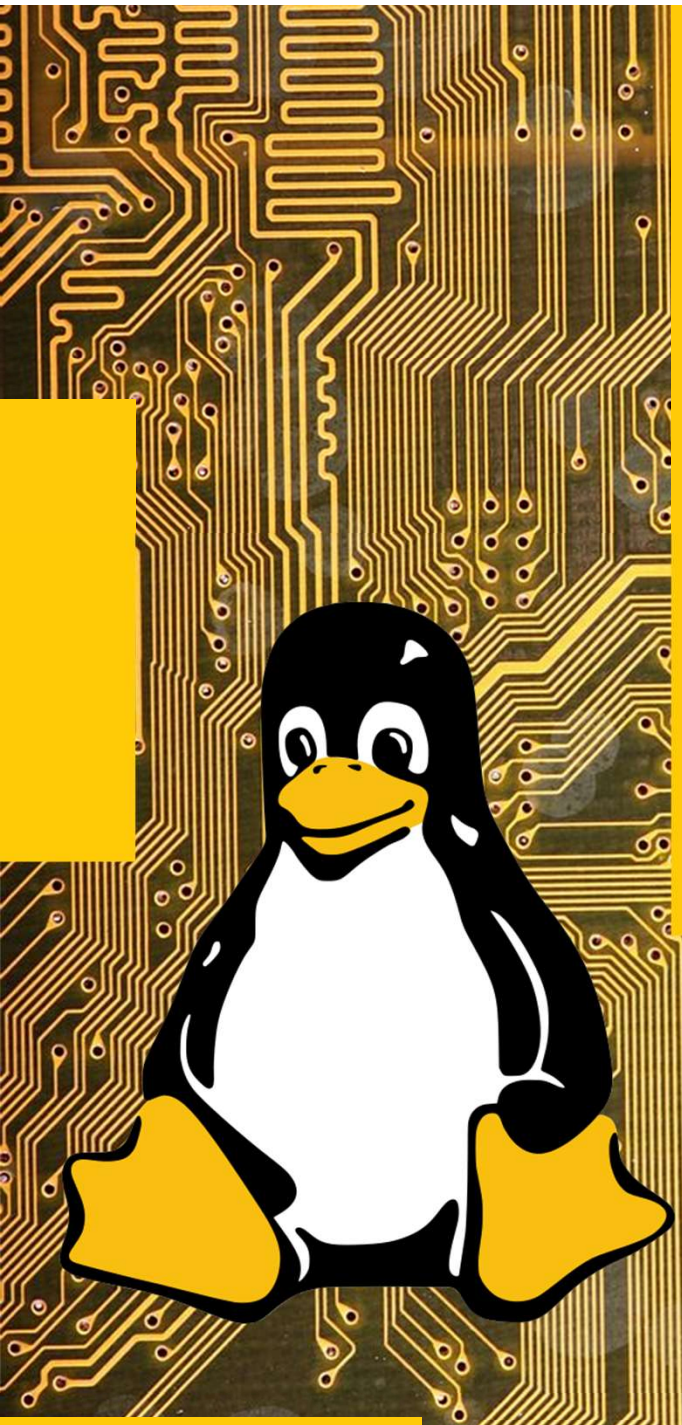


SOC 3010
OPERATING SYSTEMS

IPC II

Message Queues, Shared Memory and Mutual Exclusion

INSTRUCTOR: Jalolliddin Yusupov

WEEK IV Course Topics & Weekly Lecture Breakdown

Lecture Topic: IPC - Message Queues, Shared Memory and Semaphores (TLPI Ch 45-54)

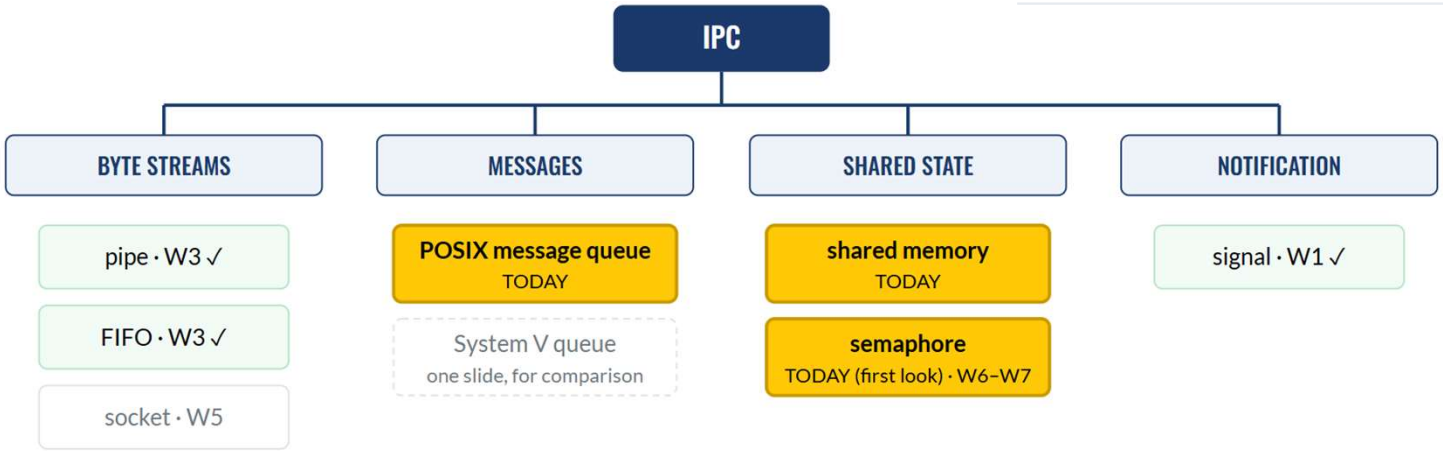
- Message queues: POSIX and System V interfaces, message priorities, the persistence of System V objects. Shared memory: creating and mapping a shared region. The lost-update problem demonstrated, and the first semaphore as the lock that protects shared memory between processes. Cleaning up IPC objects.

WHERE WE ARE GOING

Two more ways to talk: the message queue (a mailbox) and shared memory (a shared fridge) — what they are, when to use them, what they cost

The lost update: two processes add 1 a million times each, and the answer is wrong. What we need: mutual exclusion, a **mutex**

Semaphores: Dijkstra's tool, how to trace it on paper, and three ways to use it — a lock, a limit, a signal



Reading

TLPI ch. 51–54 (POSIX IPC, message queues, semaphores, shared memory).
Silberschatz §3.5, §6.6.
Manual pages:
mq_overview(7),
shm_overview(7),
sem_overview(7).

The Problem: A Pipe Does Not Know Where a Message Ends

WHAT THE WRITER DID

```
write(fd[1], "AB", 2);
write(fd[1], "AB", 2);
write(fd[1], "AB", 2);
```

WHAT THE READER MAY GET

```
n = read(fd[0], buf, 100);   /* n == 6  */
/* buf: "ABABAB" - one read, three writes glued */
```

A pipe is a **byte stream** (W3): bytes in order, and **nothing else**. It remembers the bytes, not the three separate `write` calls.

So with a pipe, you must invent

- **a separator** (every message ends with `'\n'`) — or
- **a length in front** ("the next 17 bytes are one message") — and
- **urgency**: there is none. An alarm waits behind 60 KiB of old news.

A pipe carries bytes. A message queue carries messages.

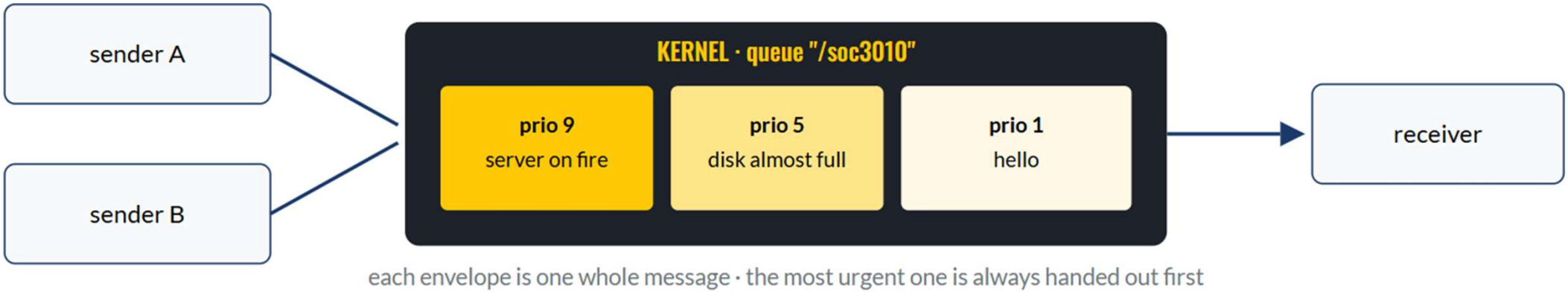
PIPE: *atomic write*

Atomic write — a write that arrives in one piece, never mixed with another writer's bytes. A pipe promises this only for writes of at most PIPE_BUF = **4096 bytes** (depending) on OS Linux. Bigger writes from two processes can interleave.

What we want instead

One send = one receive, whole or not at all — and a way to say **URGENT**.

A Message Queue Is a Mailbox the Kernel Keeps



each envelope is one whole message · the most urgent one is always handed out first

Three names, all new today

Message queue — a list of messages kept inside the kernel, with a name, that processes put messages into and take messages out of.
Message — one block of bytes sent by one call; it is always received **whole**, by one call.
Priority — a number the **sender** attaches; bigger = more urgent; the queue hands out the highest first.

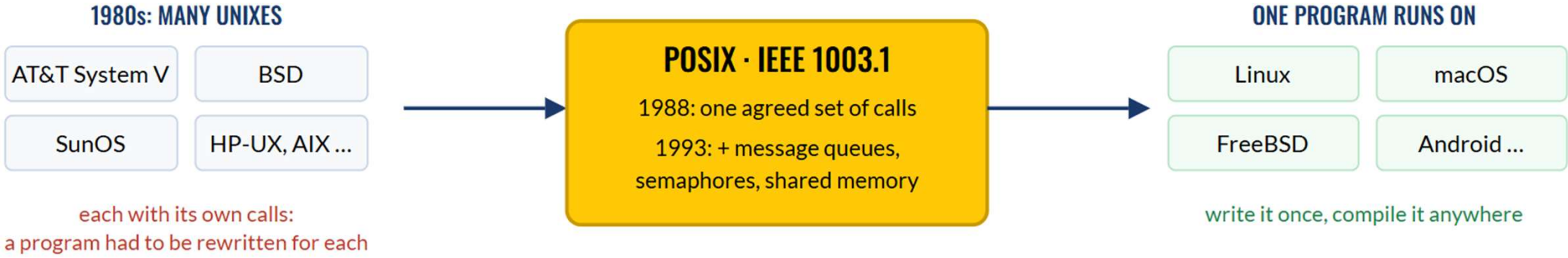
What you get for free

- **Boundaries** — no separators, no length fields.
- **Priorities** — the alarm jumps the line.
- **A (POSIX) name** — like a FIFO, unrelated processes meet at it.
- **Many senders, one receiver** — each message stays whole.

One send, one receive — the kernel counts the envelopes for you.

What Is POSIX?

POSIX — **P**ortable **O**perating **S**ystem **I**nterface (the **X** is for Uni**x**).
A **standard**: a written promise of which system calls exist,
what their names and arguments are, and how they behave.



You have used POSIX since Week 1

`fork`, `exec`, `wait`, `kill`, `sigaction`, `pipe`, `dup2`, `open`, `read`, `write`, `mkfifo` — all POSIX. So is the shell language of Week 2 and commands like `ls`, `cat`, `grep`. That is why the same C file works on Linux and on a Mac.

Windows is **not** POSIX — that is why these programs need Linux, a Mac, or WSL (Linux inside Windows).
A standard is a promise, not a guarantee

POSIX is the contract that makes a Unix program portable.

In Code: A Pipe With Envelopes

step	pipe / FIFO	POSIX message queue	the only difference
create / open	mkfifo + open	mq_open("/soc3010", …)	the name starts with /; its size (how many messages, how big) is fixed at creation
send	write	mq_send	+ a **priority** number
receive	read	mq_receive	always **one whole message**, most urgent first
close / remove	close / unlink	mq_close / mq_unlink	the queue **and its messages** survive until removed

WHAT IT LOOKS LIKE WHEN IT RUNS

```
$ ./mq_sender        # sends priorities 1, 5, 9, 5 - then exits
$ ./mq_receiver      # started later
receiver: got "server on fire" (priority 9, 15 bytes)
receiver: got "disk almost full" (priority 5, 17 bytes)
receiver: got "CPU too hot" (priority 5, 12 bytes)
receiver: got "hello" (priority 1, 6 bytes)
```

What to take away

The sender had **exited** before the receiver started — the messages waited in the kernel.
9 came out first, the two **5**s in the order they were sent, **1** last.

Same idea as a pipe — plus envelopes, plus urgency.

You will not write this in this course

Know **what** a message queue is,
that it exists, and **when** to choose it. No coding

Try It on Paper: Who Comes Out Next?

THE RULE, IN TWO LINES

1. `mq_receive` takes a message with the **highest** priority waiting.
2. Among equal priorities, the one that was **sent first** comes out first (first in, first out).

EXERCISE

Process A sends, in this order:

1	`"backup done"`, priority **0**
2	`"login failed"`, priority **3**
3	`"out of memory"`, priority **7**
4	`"login failed again"`, priority **3**

Then process B calls `mq_receive` **twice**. Process A sends `"disk dead"`, priority **7**. B receives until the queue is empty.

Answer

out of memory (7)
login failed (3)
disk dead (7)
login failed again (3)
backup done (0)

"disk dead" arrived *after* B's second receive, but it still overtakes the two older, less urgent messages.

Where this is used for real

A logger that must print **errors before chatter**; a job queue where paying customers go first. The priority belongs to the **sender**: the one who knows how urgent it is.

The Older Cousin: ~~System V Message Queues~~ vs POSIX

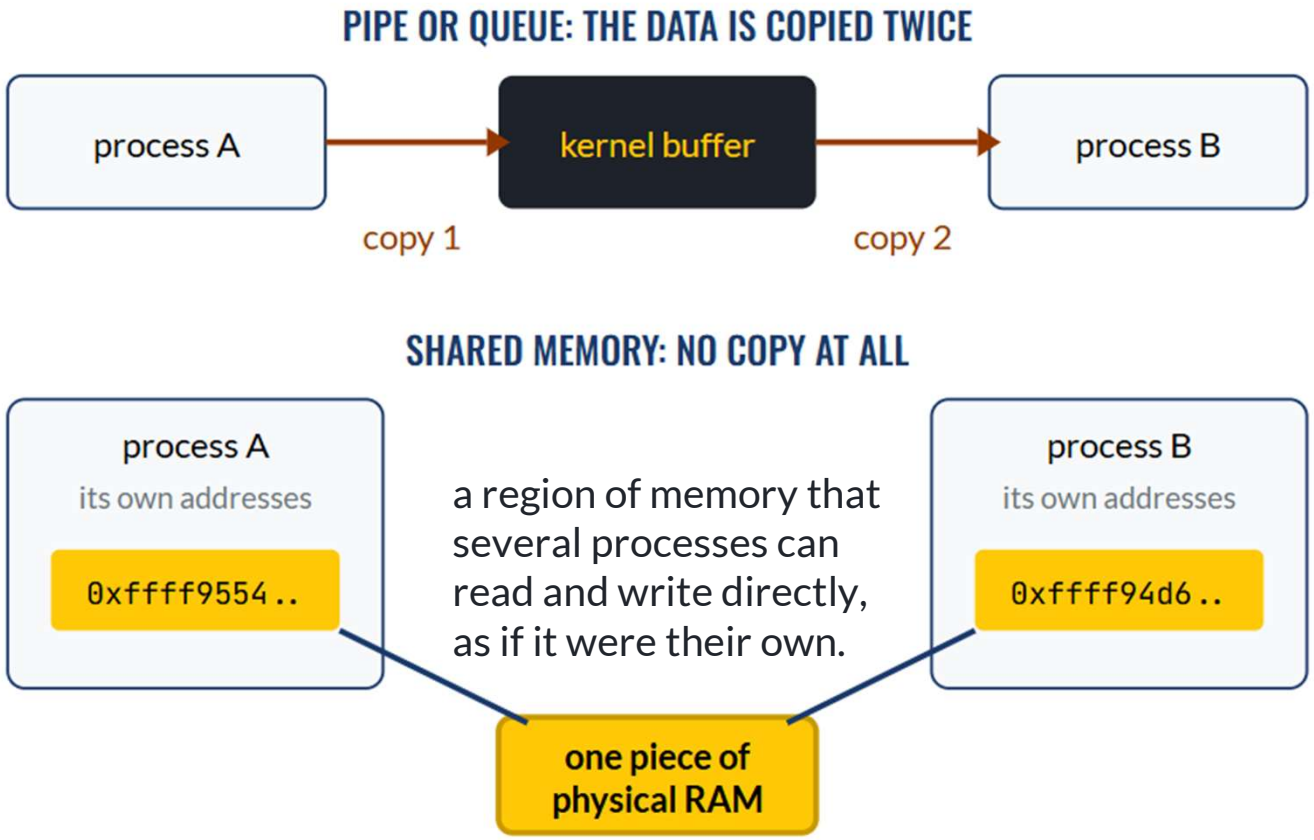
System V — the AT&T Unix of 1983 — had message queues long before POSIX. Linux still ships them and old software still uses them. In contrast to POSIX it uses an integer key instead of a /name (programs agree on the number)

question	pipe	FIFO	POSIX queue	System V queue
who can use it?	parent + children only	anyone with the path	anyone with the /name	anyone with the key
what travels?	bytes	bytes	**whole messages**	**whole messages**
who decides the order?	nobody: FIFO	nobody: FIFO	the **sender** (priority)	the **receiver** (type)
works with |, cat, echo >?	**yes**	**yes**	no	no
data survives when everybody exits?	no	no (only the name)	**yes**	**yes**
end-of-data signal?	EOF	EOF	none	none

Choose a queue when

you send **records** (a log line, a job, a command), several senders share one receiver, or some messages are **more urgent** than others.

Shared Memory: A Door in the Wall, Opened on Purpose



Remember the wall

Each process has its own address space; a pointer from one means nothing in the other.

Shared memory asks the kernel to map **the same physical memory** into both address spaces — at **different addresses**, but the same bytes.

Pipes and queues copy your data; shared memory hands out a second key to the same room.

Getting Shared Memory: One Call, Given to You

SHARED_MEM.H — ON ECLASS; READ IT ONCE, NEVER MEMORISE IT

A pre-packaged library will be provided as part of the starter code; your task is to integrate and call its functions as specified.

HOW YOU USE IT — THIS IS ALL YOU WRITE

```c
#include "shared_mem.h"

long *counter = shared_alloc(sizeof(long));
*counter = 0;
fork();       /* parent and child now share *counter */
```

The one rule

Call shared_alloc **before** fork(). The child inherits the memory, and from then on a change by one process is **seen by the other** — unlike every variable you had since, which fork copied.

One call before fork, and two processes share the same bytes.

The Price: Nobody Is Watching

With a pipe or a queue, the kernel stood in the middle of every transfer. With shared memory, after mmap the kernel **steps out**. Everything it used to do for you is now your job.

the kernel did this for pipes and queues	with shared memory
kept message boundaries	no — it is a block of bytes; you decide the layout
made the reader **wait** until data was there	no — the reader sees whatever is there **now**, finished or half-written
handed each message to **one** receiver	no — everybody sees everything
kept two writers from **mixing** their data	**no** — two processes can change the same byte at the same time

Shared memory is fast because the kernel steps out — also when you fight.

The shared fridge

Two flatmates, one fridge, no rules. Both see there is one egg left, both plan an omelette, both go to take it. Fast, convenient — and one of them is going to be wrong.
The next slide runs exactly this experiment, a million times.

The Experiment: Two Processes, One Counter

```c
/* counter_race2.c — two processes, one shared counter, NO lock */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include "shared_mem.h"
//some library industry offers us
#define N 1000000
int main(void) {
    long *counter = shared_alloc(sizeof(long));
    /* seen by parent AND child */
    *counter = 0;
    pid_t pid = fork();
    for (int i = 0; i < N; i++)
        /* BOTH processes run this loop */
        (*counter)++;
    if (pid == 0) exit(0);                  /* child: done */
    wait(NULL);                             /* parent: wait for the child */
    printf("expected %d, got %ld\n", 2 * N, *counter);
    return 0;
}
```

What it does

One long in shared memory, set to 0.
fork(): now two processes see the **same** counter.
Each adds 1.
The parent waits for the child and prints.

```
$ ./counter_race2
expected 2000000, got 1025956
$ ./counter_race2
expected 2000000, got 1073573
```

What the machine showed

About **half** of the additions disappeared. Nobody crashed, nothing printed an error, the result is nondeterministic.

The Autopsy : `counter++` Is Three Steps

For the CPU, `(*counter)++` is not one action. It is:

```
1. LOAD   the value from memory into a register
2. ADD    1 to the register
3. STORE  the register back to memory
```

The scheduler (W2) may stop a process **between any two steps**, and on a multi-core machine the two processes really run **at the same time.**

ONE UNLUCKY INTERLEAVING (COUNTER STARTS AT 41)

parent	child	in memory
LOAD 41		41
Read, add, write back — and someone else can squeeze in between.		
ADD → 42	ADD → 42	41
STORE 42		42
	STORE 42	42 — not 43



I Am Devloper
@iamdevloper

Knock knock
Race condition
Who's there?

12:07 PM - 11 Nov 2013

2,504 Retweets **1,013** Likes

38 2.5K 1.0K

"But it is one line of C!"

`(*counter)++`

Lines of C are not what the CPU runs.

Race Condition

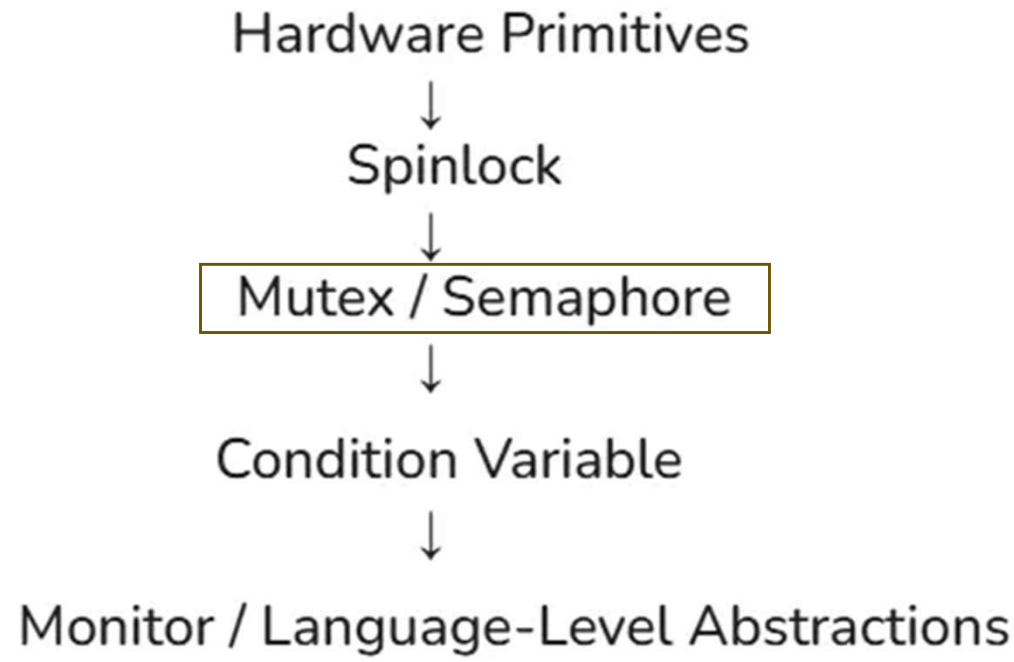
A **race condition** is an undesirable situation that occurs when a system's behavior depends on the sequence or timing of uncontrollable events, such as the execution order of multiple **threads** or **processes**.

Why They Are Dangerous

Race conditions are among the most frustrating bugs to diagnose because they are **intermittent and difficult to reproduce (where the teacher spends hours to prove the point)**.

A program might pass tests perfectly in development but fail randomly under heavy production traffic. Beyond causing **data corruption** and crashes, they represent severe **security vulnerabilities**; hackers often exploit the tiny "race window" between a security check and its execution to bypass authentication or escalate system privileges.

Mechanisms to solve Race Conditions



MUtual EXclusion

Mutual exclusion — the guarantee that while one process is inside its critical section, **no other process** can be inside it.

What We Need: Mutual Exclusion

Critical section — the lines of code that touch the **shared data**
(here: the one line (*counter)++). Only one process may be inside (locked) at a time.

A CORRECT LOCKING MECHANISM MUST GIVE US

A lock you build from ordinary variables has the same race as the thing it protects.

1 · one at a time	never two processes in the critical section together
2 · progress	if nobody is inside, somebody who wants to enter gets in
3 · no one waits forever	every process that waits gets its turn eventually
4 · no wasted CPU	a waiting process **sleeps** (state S); it does not spin in a loop checking

Why not a simple flag?

```
while (*busy == 1) ;    /* wait      */
*busy = 1;               /* I'm in    */
(*counter)++;
*busy = 0;
```

Checking the flag and setting it are **two steps** again. Both processes can see 0, both set 1, both go in. The flag has the same race as the counter — and it burns CPU while waiting.

So we need help from outside

Something that tests-and-takes in **one indivisible step**, and can put a process to sleep.
OS (with the CPU's help) can offer that.

The Mutex: A Lock Around the Critical Section

A **mutex** (short for **mut**ual **ex**clusion lock) provides two basic operations:

```
lock(m);        /* if free: take it and go on    */
                 /* if taken: SLEEP until free    */
counter++;       /* critical section              */
unlock(m);      /* free it; wake one sleeper     */
```



Every process that touches the counter follows the same rule: **lock, touch, unlock**. Then the LOAD(atomic)–ADD–STORE of one process can no longer be cut in half by the other.





parent	child	counter
lock ✓		41
LOAD 41	lock → **sleeps**	41
STORE 42	(sleeping)	42
unlock	wakes, lock ✓	42
	LOAD 42, STORE 43	43 ✓

One word, in case it is new

Atomic — indivisible: it happens completely or not at all, and nobody sees it half-done. lock checks-and-takes **atomically** — the thing our home-made flag could not do.

Example: The Mutex

```
Process P1:
    lock(L)
    print("A")
    print("B")
    unlock(L)

Process P2:
    lock(L)
    print("C")
    print("D")
    unlock(L)
```

Which of the following outputs are possible?

A B C D ⟵

C D A B ⟵

A C B D

C A D B

Next: What if we have two locks?

```
Process P1:
    lock(L1)
    lock(L2)
    print("P1 done")
    unlock(L2)
    unlock(L1)

Process P2:
    lock(L2)
    lock(L1)
    print("P2 done")
    unlock(L1)
    unlock(L2)
```

Homework

1. How many different outputs are possible?

```
Process P1:          Process P2:          Process P3:

lock(L)              lock(L)              lock(L)
print("A")           print("C")           print("E")
print("B")           print("D")           print("F")
unlock(L)            unlock(L)            unlock(L)
```

2. Can the output be ACBD?

```
Process P1:          Process P2:

lock(L1)             lock(L2)
print("A")           print("C")
print("B")           print("D")
unlock(L1)           unlock(L2)
```



Questions?